Lab 5. Pseudo Random Number Generation in C

Jianhua Liu

Spring 2026

Contents

5.1 Introduction . . . 1
5.1.1 Purpose of the lab . . . 1
5.1.2 Background info about pseudo-random noise generation . . . 2
5.1.3 Learning outcomes . . . 3
5.2 Project preparation . . . 3
5.2.1 PRN generation . . . 3
5.2.2 Test functions . . . 5
5.2.3 An App using the PRN functions driven by a timer for turning on/off an LED . . . 7
5.3 Project creation tasks (20 pts) . . . 8
5.4 Programming task (PT) of the project (70 pts) . . . 9
5.4.1 PT 1 (15 pts): Program an initialization function . . . 9
5.4.2 PT 2 (15 pts): Program a tap-mask generation function . . . 9
5.4.3 PT 3 (35 pts): Program the main PRN generation function . . . 9
5.4.4 PT 4 (5 pts): Program the App . . . 10
5.5 Submission of the work (10 pts) . . . 10
5.6 Appendix A: Diagram of the PRN generator in another format . . . 10
5.7 Hints for the programming . . . 12

5.1 Introduction

5.1.1 Purpose of the lab

The purpose of this lab is to develop a pseudo-random noise (PRN) sequence generator using a microcontroller. PRN sequences play a crucial role in various systems, such as GPS, where they are used to encode signals from multiple satellites. In this lab, we will implement a PRN sequence generator in software, following the test-driven development (TDD) approach, and use it to control an LED in a seemingly random blinking pattern.

Do not start the lab until you read the entire document and fully understand the requirements and instructions.

5.1.2 Background info about pseudo-random noise generation

In many applications, PRN sequences are generated using feedback shift registers, where certain bits of the register are tapped and combined using an exclusive OR (XOR) operation. The output of the XOR will be fed back to the shift register from the least significant bit (LSB), continuing the sequence generation. Note that the shift register shifts data in the direction from LSB to most significant bit (MSB).

A commonly used PRN generator is represented by the polynomial $1 + x^3 + x^{10}$. This polynomial defines the feedback connections within a 10-bit shift register. The key details include:

- The PRN generator's state is a non-zero 10-bit value stored in a shift register. In this lab, we follow a convention where the register bits are labeled Bit1 to Bit10 from LSB to MSB.
- The feedback bit is computed using XOR on Bit3 and Bit10, corresponding to x^3 and x^{10}, respectively. This feedback is then shifted into Bit1 with each clock cycle.
- The output of the PRN generator is the bit that shifts out from MSB.

A diagram illustrating the PRN generator operation over 12 clock cycles is provided in Figure 1. If this notation is unfamiliar, an alternative notation is available in Figure 2 at the end of this handout.

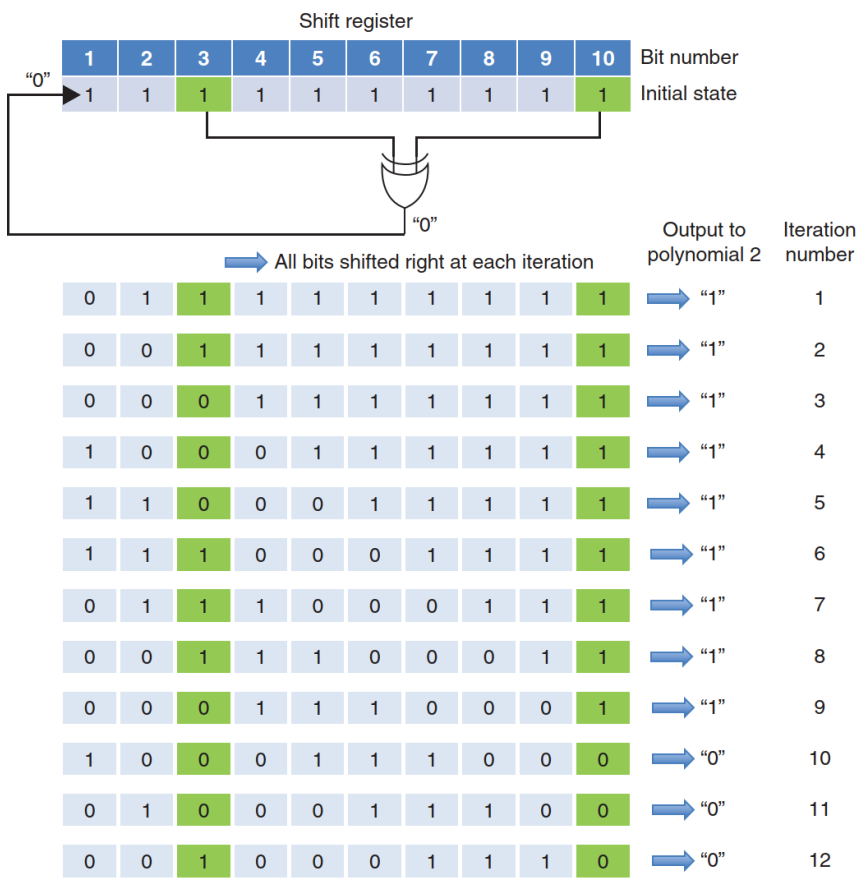

Figure 1: A PRN generator

5.1.3 Learning outcomes

This lab builds upon previous coursework and provides hands-on experience in implementing and testing PRN generators, preparing students for more advanced applications in digital communication and embedded systems.

By completing this lab, students will:

- Gain a deeper understanding of PRN sequence generation and its role in real-world applications like GPS.
- Implement a PRN generator using C and verify its functionality through test-driven development (TDD).
- Use PRN sequences to control LED blinking, reinforcing concepts of randomness in embedded systems.
- Strengthen programming skills in C while working with bitwise operations and shift registers.

5.2 Project preparation

5.2.1 PRN generation

There are many different approaches to implement a PRN generator in C. Here, we consider one of the easy-to-implement approaches. Although the implementation is easy, getting to know all the background information can lead to frustration. Just relax; with a few reading and playing, you will be there.

In this project, you will write three C functions to implement this generator. These functions are defined in `mp_prn_c.c`, shown below:

```c
#include "mp_prn_c.h"
#include "mp_sum_of_1s.h"

int poly1[] = {3, 4};
int poly2[] = {3, 10};
int poly_n = 2;
uint32_t prn_reg;

static uint32_t reg_mask;   // mask to limit PRN register value
static uint32_t tap_mask;   // mask for taps (each tap is 1)

// Initializing the PRN register---dereferenced prn_reg
__attribute__((weak))
uint32_t mp_prn_ini_c(uint32_t *prn_reg, int *poly, int poly_n) {
    // write your code here.
    return a_reg_mask;
}

// Building the tap mask from poly and poly_n
__attribute__((weak))
uint32_t mp_prn_tap_mask_c(int *poly, int poly_n) {
    // write your code here.
    return a_tap_mask;
}

// Generating the output bit and updating prn_reg
__attribute__((weak))
bool mp_prn_gen_c(uint32_t *prn_reg) {
    // write your code here.
    return true;
}
```

In the above code, we first define two polynomials, `ploy1` and `poly2`, each having two feedback taps, indicated by `poly_n`.

We then define the shift register, a global variable named `prn_reg`.

We will also define two private variables, `reg_mask` and `tap_mask`. The purposes of defining these variables will be discussed below.

`mp_prn_ini_c` is the C function used to initialize the value of the shift register.

- To be able to change the value of `prn_reg` as an argument of the function, we need to use pointer `*prn_reg`.
- This function also needs to use the pointer of the polynomial together with the value of the number of elements in the polynomial.
- The convention is that the initial value of the shift register should be 1 for each effective bit. For example, for the above polynomial [3, 10], the value of the initialize value of register should be `0x03FF`, as discussed below.

During the normal operation to generate PRNs, the value of this shift register should be shifted from LSB to MSB, as required by the operation. To keep the value of the register to a given number of bits, we have defined a private variable `reg_mask`, which is the mask of the given number of bits. For the case of [3, 10], we only keep a total of 10 bits. The value of `reg_mask` should be the same as the initial value of the shift register, which should be 10 1's, leading to `0x3FF`. This is why return `a_reg_mask` from this initialization function.

`mp_prn_tap_mask_c` is the function to obtain the tap mask, expressed using variable `tap_mask`, used for tapping the bits from the shift register used for feedback. With this tap mask, we just need to use a single bitwise AND operation between `tap_mask` and `prn_reg` to tap all the bits to obtain the feedback. For example, for the above polynomial [3, 10], the value of the tap mask should be `(1<<2)+ (1<<9)` or `(1<<2)| (1<<9)`. This function returns `a_tap_mask` according to the input polynomial and the number of elements of the polynomial. With `tap_mask`, we can easily tap the corresponding bits in the third function, which is discussed below.

`mp_prn_gen_c` is the clock-based PRN generation function. With each call, it works as if it is driven by a clock to shift every bit from LSB to MSB. This function will return the output bit and change the value of the shift register in place by passing the argument via pointer instead of value. We need to do the following in this function:

- Tap the bits using the above-mentioned tap mask, `tap_mask`.
- For the taped bits, add them together by using the provided `mp_sum_of_1s` function in the `lib` folder. Just keep the LSB of this summation, which is equivalent to the XOR of all the bits. (You can verify this operation.)
- Shift the bits from LSB to MSB by one bit as if this is driven by a clock.
- Feedback the XOR value to the LSB of the shift register.
- Make sure the number of bits is still the same as the maximum value of the polynomial elements. This can be done by ANDing `prn_reg` with `reg_mask`.

Note that we have used suffix `c` in the function names as we can program these function in assembly as well.

We have discussed the arguments of the above functions above. It does not hurt to summarize them here again:

- `uint32_t *prn_reg`: pointer of the shift register. This register is the 10-bit shift register in Figure 1. To be able to save its value in the function call, we need to use a pointer instead of value. We use an `uint32_t` data type so that we can use our functions in the applications where the shift register can be up to 31 bits. Note that the variable holding this shift register is defined by the caller so that its value can be easily accessed for testing purposes.
- `int *poly`: the polynomial values of the PRN generator. For the above example polynomial, we have [3, 10]. The array holding the polynomial is defined by the caller so that it is easy to change their values.
- `int poly_n`: the number of elements in `poly[]`. For polynomial [3, 10], `poly_n` is 2. Using the pointer for the polynomial array and the number of elements of the polynomial, we can easily find the number of effective bits of the shift register, which is 10 for the case of polynomial [3, 10].

Note that you need to consider the generic cases where the polynomial may have more items. You need to start with the following header file `mp_prn_c.h`, which has the **definitions of the arguments of the functions**:

```c
#pragma once

#ifdef __cplusplus
extern "C" {
#endif /* __cplusplus */

#include <stdint.h>
#include <stdbool.h>

// Initializing the PRN register---dereferenced prn_reg
// The following are the input and output of the PRN functions:
//    Input:
//      uint32_t *prn_reg: the pointer of the PRN register.
//      int *poly: the polynomial values of the PRN generator.
//          For a given example, this is [3, 10]
//      int poly_n: the number of elements in poly[].
//    Output:
//      uint32_t reg_mask: the register mask
uint32_t mp_prn_ini_c(uint32_t *prn_reg, int *poly, int poly_n);

// Building the tap mask from poly and poly_n.
//    Output:
//      uint32_t tap_mask: the tap mask. For the example with poly [3, 10],
//          this tap mask is (1<<2) | (1<<9).
uint32_t mp_prn_tap_mask_c(int *poly, int poly_n);

// Generating the output bit and updating the PRN register---reg.
//    Output:
//      bool output: the output bit.
bool mp_prn_gen_c(uint32_t *prn_reg);

#ifdef __cplusplus
}
#endif /* __cplusplus */
```

5.2.2 Test functions

The above functions will be tested using Unity, as detailed below in the `main4unit_test.c` file:

```c
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>
```

```c
#include "unity.h"
#include "mp_prn_c.h"

extern int poly1[];
extern int poly2[];
extern int poly_n;
extern uint32_t prn_reg;

//-----------------------------------------------
void test_prn_ini_c(void) {
    uint32_t exp[] = {0x000F, 0x03FF};
    uint32_t act[2];
    mp_prn_ini_c(&act[0], poly1, poly_n);
    mp_prn_ini_c(&act[1], poly2, poly_n);
    TEST_ASSERT_EQUAL_INT32_ARRAY(exp, act, 2);
}

//-----------------------------------------------
void test_prn_tap_mask_c(void) {
    uint32_t exp[] = {(1<<2) + (1<<3), (1<<2) + (1<<9)};
    uint32_t act[2];
    act[0] = mp_prn_tap_mask_c(poly1, poly_n);
    act[1] = mp_prn_tap_mask_c(poly2, poly_n);
    TEST_ASSERT_EQUAL_INT32_ARRAY(exp, act, 2);
}

//-----------------------------------------------
void test_prn_gen_for_prn_poly1_c(void) {
    uint8_t exp[] = {1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0};
    uint8_t act[15];
    mp_prn_ini_c(&prn_reg, poly1, poly_n);
    mp_prn_tap_mask_c(poly1, poly_n);
    for (int i = 0; i < 15; i++) {
        act[i] = mp_prn_gen_c(&prn_reg);
    }
    TEST_ASSERT_EQUAL_INT8_ARRAY(exp, act, 15);
}

//-----------------------------------------------
void test_prn_gen_for_reg_poly1_c(void) {
    uint32_t exp[] =
            {0xE, 0xC, 8, 1, 2, 4, 9, 3, 6, 0xD, 0xA, 5, 0xB, 7, 0xF};
    uint32_t act[15];
    mp_prn_ini_c(&prn_reg, poly1, poly_n);
    mp_prn_tap_mask_c(poly1, poly_n);
    for (int i = 0; i < 15; i++) {
        mp_prn_gen_c(&prn_reg);
        act[i] = prn_reg;
    }
    TEST_ASSERT_EQUAL_INT32_ARRAY(exp, act, 15);
}

//-----------------------------------------------
void test_prn_gen_for_prn_poly2_c(void) {
    uint8_t exp[] = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1};
    uint8_t act[15];
    mp_prn_ini_c(&prn_reg, poly2, poly_n);
    mp_prn_tap_mask_c(poly2, poly_n);
    for (int i = 0; i < 15; i++) {
        act[i] = mp_prn_gen_c(&prn_reg);
```

```c
    }
    TEST_ASSERT_EQUAL_INT8_ARRAY(exp, act, 15);
}

//-----------------------------------------------
void test_prn_gen_for_reg_poly2_c(void) {
    uint32_t exp[] =
            {0x3FE, 0x3FC, 0x3F8, 0x3F1, 0x3E3, 0x3C7, 0x38E,
            0x31C, 0x238, 0x71, 0xE2, 0x1C4, 0x389, 0x313, 0x227};
    uint32_t act[15];
    mp_prn_ini_c(&prn_reg, poly2, poly_n);
    mp_prn_tap_mask_c(poly2, poly_n);
    for (int i = 0; i < 15; i++) {
        mp_prn_gen_c(&prn_reg);
        act[i] = prn_reg;
    }
    TEST_ASSERT_EQUAL_INT32_ARRAY(exp, act, 15);
}

void mp_unity(void) {
    UNITY_BEGIN();

    RUN_TEST(test_prn_ini_c);
    RUN_TEST(test_prn_tap_mask_c);

    RUN_TEST(test_prn_gen_for_prn_poly1_c);
    RUN_TEST(test_prn_gen_for_reg_poly1_c);

    RUN_TEST(test_prn_gen_for_prn_poly2_c);
    RUN_TEST(test_prn_gen_for_reg_poly2_c);

    UNITY_END();

    while (1);
}
```

Note that the above test cases can be hand-calculated or be generated by a higher-level language, such as Python.

5.2.3 An App using the PRN functions driven by a timer for turning on/off an LED

When all the PRN functions have passed their unit tests, we are ready to use these functions in an App. Here, we can use Timer 4's callback function to drive the generation of the PRN to control LED4. With this, we can see the pseudo random change of the intensity (or blinking) of LED4.

Note that this App only works on real MPB as simulation of the LED change is not easily doable.

Here are more details of the App:

- The PRN generator is driven by Timer 4. In the callback function of Timer 4, we will call `mp_prn_gen_c` to generate a new random Boolean output. We will then use this output to turn on/off LED4.
  - The clock rate is 100 MHz. We need to do a quick calculation of the PSC and ARR so that we can have an interrupt rate at about 100 Hz. A quick calculation leads to the choice of both PSC and ARR being 1000 for an interrupt rate of 100 Hz.
  - The above are all set up in the provided `dk5u_prn_generation_in_c_g431n32.ioc` file.

We can change the value of either PSC or ARR to change the interrupt rate. For convenience, we can just change the value of PSC only.

- The expected value of PSC of Timer 4 can be passed to the MCU using UART. When the value of PSC is low, the interrupt rate is high, and we can see the change of the flickering intensity of LED4; when the value of PSC is high, the interrupt rate of low, and we can see the random blinking.
  - The value of PSC can be modified so that we can change the intensity or blinking rate of the LED.
  - The range of the value of PSC is set up to be between 100 and 10000 in the callback function of the UART.
- The initialization of the PRN generator, the running of the `mp_prn_ini_c` and `mp_prn_tap_mask_c` functions, will be done in the initial state of the UART callback function. The timer is started here as well.

5.3 Project creation tasks (20 pts)

As we mentioned before, this project is a good example of TDD. To this end, we want to be able to develop the individual functions using simulator and the real App using the read MPB. So, we need to create projects for both F412Dsc and G431N32.

See the enclosed `dk5u_prn_generation_in_c-base-files.zip` file for all the files you need to get started. Unzip this file and create the following folder under `proj_mp`:

```
dk5u_prn_generation_in_c
    dk5u_prn_generation_in_c_f412dsc
    dk5u_prn_generation_in_c_g431n32
    lib
    renode
    src
    test
```

Specifically, we need to do the following:

- Generate the projects using CubeMX for both F412Dsc and G431N32 using the `.ioc` files in the above folders. Embed the hook function `mp_main()` for both projects.
- In CubeIDE, we need to create the `lib_src` folder for both project. We also need to include the `lib` and `src` folders in the include path.

  For the `dk5u_prn_generation_in_c_f412dsc` project, we only link the following files to the `lib_src` folder since we only need to run unit test:

  ```
  _mp_main.c
  mp_prn_c.c
  mp_sum_of_1s.c
  mp_uart_redirect.c
  test_dk5u_prn_generation_in_c.c
  unity.c
  ```

  For the `dk5u_prn_generation_in_c_g431n32` project, we need to link the following additional files to the `lib_src` folder since we will build App (debug) configuration as well:

```
dk5u_prn_generation_in_c_app.c
dk5u_timer_fn_n_callback.c
dk5u_uart_callback.c
```

- In CubeIDE, we need to create the Unity build configuration for both F412Dsc and G431N32. Node that we need to create the `UNIT_TEST` symbol for the Unity build configuration.

Note that running the code using Renode can be quicker, but if you run into trouble, you need to use the `Unity` build configuration of G431N32 to debug the code.

5.4 Programming task (PT) of the project (70 pts)

We have four PTs for this project. The first three are about developing functions with TDD, the final one is about the application of the tested functions.

For unit test, we need to perform the following three PTs, all hosted in the `mp_prn_c.c` file.

5.4.1 PT 1 (15 pts): Program an initialization function

Program the following initialization function:

```
void mp_prn_ini_c(uint16_t *reg, int *poly, int poly_n);
```

to pass the following test:

```
RUN_TEST(test_prn_ini_c);
```

Note that you need to extract the needed info from the arguments, which means you cannot assume `poly_n` is 2.

Take a screenshot of the running results. This is **Artifact 1**.

5.4.2 PT 2 (15 pts): Program a tap-mask generation function

Program the following tap mask generation function:

```
uint16_t mp_prn_tap_mask_c(int *poly, int poly_n);
```

to pass the following test:

```
RUN_TEST(test_prn_tap_mask_c);
```

Take a screenshot of the running results. This is **Artifact 2**.

5.4.3 PT 3 (35 pts): Program the main PRN generation function

Program the following PRN generation function:

```
bool mp_prn_gen_c(uint16_t *reg);
```

to pass the following tests:

```
    RUN_TEST(test_prn_gen_for_prn_poly1_c);
    RUN_TEST(test_prn_gen_for_reg_poly1_c);

```

```
    RUN_TEST(test_prn_gen_for_prn_poly2_c);
    RUN_TEST(test_prn_gen_for_reg_poly2_c);
```

If you need hints as how to program the above functions, please see the last page of this handout.

Take a screenshot of the running results. This is **Artifact 3**.

5.4.4 PT 4 (5 pts): Program the App

To run the App which uses Timer 4 to drive the PRN generator function, we need to use Timer 4's interrupt handler or a callback function. Normally, we want to find an appropriate callback function and redefine it to add our user code. Yet, timers have a large number of different functionalities, which makes finding the appropriate callback function confusing. An easy way is just to **embed our own Timer 4 callback function**. This is done in `stm32g4xx_it.c`, where we need to change the handler of Timer 4 to the following:

```
void TIM4_IRQHandler(void)
{
  /* USER CODE BEGIN TIM4_IRQn 0 */
  void mp_TIM_Callback(void);
  mp_TIM_Callback();
  /* USER CODE END TIM4_IRQn 0 */
  HAL_TIM_IRQHandler(&htim4);
  /* USER CODE BEGIN TIM4_IRQn 1 */

  /* USER CODE END TIM4_IRQn 1 */
}
```

After this, the App should work.

5.5 Submission of the work (10 pts)

Here is a brief listing of the results for the lab:

- Code snippets of the three functions you have programmed.
- Screenshots of printout from the `Unity` build for both boards showing that you have passed all the tests.
- Screenshot of the printout from the App build of G431N32 board.

Please team up with another student to work on the project together. **You can submit a single copy report as a team**; if you choose to do so, please list the name of both team members.

The report and zip file names should be something like:

- `dk5u-report-lastname-firstname.pdf`.
- `dk5u-proj-lastname-firstname.zip`.

5.6 Appendix A: Diagram of the PRN generator in another format

Here, we show 4 taps (with two dotted lines) and a summation of the values from these taps. A *modulo with 2* operator follows the summation so that the output will be either 0 or 1, which will be fed to b_0 in the next clock.

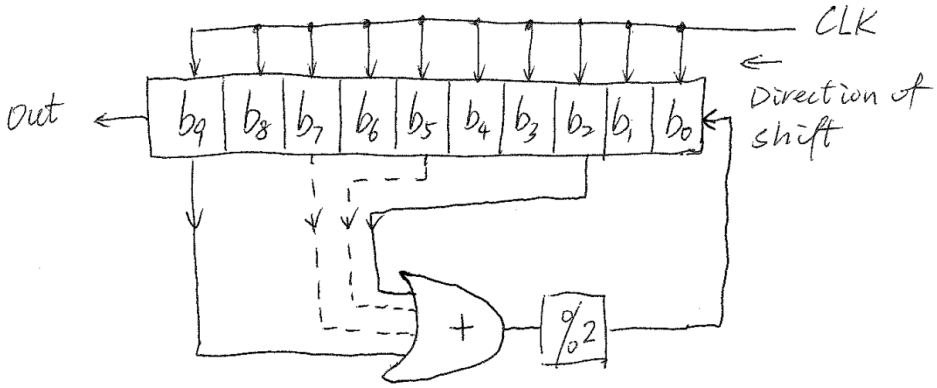

Figure 2: A PRN generator in the normal notation.

5.7 Hints for the programming

Function 1 is simple, as we have been doing similar operations a lot. Once you fully understand what is needed, the programming is straightforward.

Function 2 is not hard either; the key is to understand what is asked for. Study the test cases and fully understand them. After that, the programming is straightforward as well.

Make sure you need to save the needed values of the private variables.

Function 3 is a bit tricky; still, it is only a few lines of code. The key is to fully understand the principle of PRN generation according to the diagram we provided. Then, do the following step by step:

- Get the **value** of the shift register by dereferencing it.
- Tap the bits of the shift register's *value* using the tap mask. This is a bitwise AND operation.
- Count the sum of 1s in the tapped bits. If this is an even number, then we feedback 0; otherwise, we feedback 1. This can be done using a simple AND with `1U`. Be careful with where your feedback goes to.
- Shift the MSB of the PRN register out (note that this is the not the MSB of the 32-bit number; it is the MSB of the n-bit PRN register) and use it as the return. There are many ways to implement this.
- Make sure the number of bits in the shift register is still limited by the highest value in array `poly`. This can be done easily using `reg_mask`. Update the register.